AIOps Demo 部署手册

AWS DevOps Agent × 飞书应用机器人 · 分步部署、验证与清理

项目	内容
版本	1.0（2026-10-22）
部署区域	us-east-1
部署内容	一个 CloudFormation 栈 `aiops-demo` + DevOps Agent 侧配置 + 一个飞书企业自建应用
预计耗时	首次约 60–90 分钟（含飞书应用审核等待），其中栈部署约 10–15 分钟
配套文档	《01-方案说明》《02-设计文档》

命令均在项目 `aiops-aws-demo` 的根目录下执行；`<...>` 为需要替换的值。

0. 部署总览

阶段	内容	在哪里操作	章节
准备	账号、权限、本机工具	本机	1
A	创建 Agent Space 与 HMAC Webhook	AWS 管理控制台	2
B	创建飞书应用、开通机器人能力与权限	飞书开发者后台	3
C	部署 AWS 资源	本机	4
D	录入凭证	本机	5
E	飞书事件订阅（长连接）、发布版本、拉群	飞书开发者后台 / 飞书客户端	6
F	绑定演示群	本机	7
G	注册 MCP	本机	8
H	上传 Skill	DevOps Agent Web App	9
I	验证与演练	飞书 / 本机	10

顺序上有两处依赖：飞书“使用长连接接收事件”要求保存时客户端已经在线，所以第 6 章必须在第 4、5 章之后；绑定演示群需要先在群里 @ 一次机器人拿到群 ID，所以第 7 章在第 6 章之后。

1. 准备工作

1.1 AWS 账号与权限

- 一个可用于演示的 AWS 账号，使用 us-east-1 区域。
- 部署所用的 IAM 身份需要能创建 IAM 角色（模板使用命名角色，需 `CAPABILITY_NAMED_IAM`），并能管理 CloudFormation、Lambda、DynamoDB、SNS、CloudWatch、EventBridge、Secrets Manager、SSM、S3、Cognito、Amazon Bedrock AgentCore、ECS、EC2（安全组、查询 VPC 与

子网）、CloudWatch Logs，以及 DevOps Agent（`aidevops:*` 中的注册与关联操作）。演示账号中通常直接使用管理员权限。

- 账号中存在默认 VPC（长连接客户端默认部署在默认 VPC 的公有子网，仅出站）。没有默认 VPC 时，见 4.3 的网络参数。
- 账号或组织的安全策略允许：注册自定义 MCP Server、函数 URL 使用 `AWS_IAM` 认证、Fargate 任务带公网 IP 出站。

1.2 本机工具

工具	要求	检查命令
AWS CLI	v2，包含 `devops-agent` 命令（2.36.47 已验证）	`aws --version`、`aws devops-agent help`
Python	3.12	`python3.12 --version`
其他	bash、zip、可访问 PyPI（打包时下载 linux/arm64 依赖）	`zip -v`

本机为 x86 或 arm 架构均可，依赖按 linux/arm64 wheel 下载，不需要 Docker。

1.3 配置 AWS 凭证

```
aws configure --profile <演示账号 profile>        # 或：aws sso login --profile <profile>
export AWS_PROFILE=<演示账号 profile>
export AWS_REGION=us-east-1
aws sts get-caller-identity                          # 确认账号 ID 正确
```

后续所有命令都在设置了 `AWS_PROFILE` 的同一个终端中执行。凭证到期（如 SSO 会话）后需重新登录。

1.4 飞书侧准备

- 确认企业使用飞书（`open.feishu.cn`）还是 Lark（`open.larksuite.com`）租户，应用必须建在实际使用的租户中。
- 操作人需要有创建企业自建应用的权限；企业开启了应用审核时，提前约好管理员审批版本发布。
- 准备一个演示用飞书群，操作人是群成员并可添加机器人。

2. 阶段 A：DevOps Agent 控制台

2.1 创建 Agent Space

1. 登录 AWS 管理控制台，切换到 **美国东部（弗吉尼亚北部）us-east-1**，打开 **AWS DevOps Agent**。
2. 选择 **Create Agent Space**，名称填 `aiops-demo`，描述可填“AIOps Demo”。

3. IAM 角色选择自动创建（控制台会创建 `DevOpsAgentRole-AgentSpace` 等角色，Agent 用它只读访问账号资源）。
4. 当前账号会自动作为主数据源（primary source）关联，无需额外操作。
5. 开启 **Web App**，选择 **Auto-create a new AWS DevOps Agent role (recommended)**。
6. 创建完成后，在 Agent Space 详情中记下 **Agent Space ID**（UUID 格式）。

参考：创建 Agent Space

把回复语言设为中文（控制台中如有语言选项也可直接选择）：

```
export AGENT_SPACE_ID=<Agent Space ID>
aws devops-agent update-agent-space --agent-space-id "$AGENT_SPACE_ID" --locale zh-CN
aws devops-agent get-agent-space --agent-space-id "$AGENT_SPACE_ID" --query
agentSpace.locale   # 期望 "zh-CN"
aws devops-agent get-operator-app --agent-space-id "$AGENT_SPACE_ID" --query
operatorAppUrl     # Web App 地址
```

检查点：`get-operator-app` 返回 `https://<Agent Space ID>.aidevops.global.app.aws`。

2.2 创建 HMAC Webhook

1. 打开 Agent Space `aiops-demo` → **Capabilities** 选项卡 → **Agent Space Webhook** 表格。
2. 添加 Webhook，认证方式选 **HMAC**。
3. 复制 **Webhook URL** 和 **签名密钥**，妥善保存。**签名密钥只显示这一次**，第 5 章录入时需要。

参考：通过 Webhook 触发调查。每个 Agent Space 只能有一个 HMAC Webhook；密钥丢失时删除后重新创建。

检查点：

```
ASSOC=$(aws devops-agent list-associations --agent-space-id "$AGENT_SPACE_ID" \
  --query "associations[?serviceId=='event_channel'].associationId" --output text)
aws devops-agent list-webhooks --agent-space-id "$AGENT_SPACE_ID" --association-id
"$ASSOC" \
  --query 'webhooks[].[webhookType,webhookUrl]' --output table          # 期望出现一条 hmac
```

3. 阶段 B：创建飞书应用

本阶段在飞书开发者后台完成，可与第 4 章并行。界面名称以飞书官方文档为准，如界面略有调整，权限可直接按 `im:` 开头的权限标识搜索。

3.1 创建企业自建应用

1. 打开 飞书开发者后台（Lark 租户为 `https://open.larksuite.com/app`），选择 **创建企业自建应用**。
2. 填写应用名称（如“Devops Agent”，即群里显示的机器人名称）、应用描述、图标，确认创建。

3.2 记录应用凭证

应用详情页 → **凭证与基础信息** → **应用凭证**，记下 **App ID**（`cli_` 开头）与 **App Secret**，第 5 章录入。

3.3 添加机器人能力

添加应用能力 → **机器人** → **添加**。

3.4 开通权限

权限管理 → **开通权限**，搜索并开通以下两项：

权限名称	权限标识	用途
获取群组中用户@机器人消息	`im:message.group_at_msg:readonly`	接收群里 @机器人 的消息
以应用的身份发消息	`im:message:send_as_bot`	发送告警 / 根因卡片、回复问答

不需要开通“获取群组中所有消息”（`im:message.group_msg`，敏感权限）。

参考：飞书：接收消息事件（权限要求）、飞书：发送消息

> 事件订阅（3.4 之后的“事件与回调”）此时**先不配置**：保存长连接需要客户端在线，放到第 6 章。

4. 阶段 C：部署 AWS 资源

4.1 部署命令

```
cd aiops-aws-demo
AGENT_SPACE_ID="$AGENT_SPACE_ID" ENABLE_CHAT_BOT=true ./deploy.sh
# Lark 租户：再加 FEISHU_BASE_URL=https://open.larksuite.com
```

`deploy.sh` 依次完成：

1. 查询账号与 Web App 地址；
2. 打包 Lambda 公共层（锁定版本的 boto3）；
3. 创建制品桶 `aiops-demo-artifacts-<账号>-us-east-1`（开启 S3 阻止公有访问）并上传代码包；
4. 构建 es-mock 与飞书长连接客户端的 linux/arm64 代码包（按内容哈希命名，未变化时跳过）；
5. 选定默认 VPC 的默认子网（排除不提供 Fargate 的 `use1-az3`）；

6. 部署 CloudFormation 栈 `aiops-demo`，并打印栈输出。

4.2 期望输出

```
==> account <账号>, region us-east-1, stack aiops-demo
==> chat bot: true, cards via: custom bot webhook
...
Successfully created/updated stack - aiops-demo
```

首次部署时 `cards via` 显示 `custom bot webhook` 是正常的，第 7 章绑定群 ID 后会变为 `app bot`。主要栈输出：

输出	用途
`NexusMcpUrl`	nexus-mock 端点（第 8 章自动读取）
`EsMcpRuntimeArn`	es-mock 运行时（第 8 章自动读取）
`McpOAuthTokenUrl` / `McpOAuthClientId` / `McpOAuthScope`	es-mock 的 OAuth 配置
`FeishuWsService`	长连接客户端的 ECS 服务名
`ChatWorkerFunctionName`	问答 Worker
`WebhookSecretName` / `FeishuSecretName`	第 5 章写入的两个 Secret

4.3 可选参数

环境变量	默认	说明
`ENABLE_CHAT_BOT`	沿用已部署值，首次为 `false`	`true` 部署长连接客户端与问答 Worker
`FEISHU_CHAT_ID`	沿用已部署值	演示群 ID（第 7 章）；传空值 `FEISHU_CHAT_ID=` 表示清除
`FEISHU_BASE_URL`	`https://open.feishu.cn`	Lark 用 `https://open.larksuite.com`
`FEISHU_WS_DESIRED_COUNT`	`1`	`0` 停用长连接任务
`FEISHU_WS_VPC` / `FEISHU_WS_SUBNETS`	默认 VPC / 默认子网	无默认 VPC 时指定；子网需有到互联网的出站路由
`TRAFFIC_ENABLED`	`true`	`false` 停用模拟流量
`ENABLE_AGENTCORE_MCP`	`true`	`false` 不部署 es-mock
`STACK_NAME`	`aiops-demo`	栈名

检查点：

```
aws logs tail /aws/lambda/aiops-demo-traffic --since 5m      # 期望 {"requests": 20, "ok":
20, "failed": 0}
aws ecs describe-services --cluster aiops-demo --services aiops-demo-feishu-ws \
  --query 'services[0].[runningCount,deployments[0].rolloutState]'   # 期望 [1,
"COMPLETED"]
aws logs tail /ecs/aiops-demo-feishu-ws --since 10m
# 此时尚未录入飞书凭证，日志显示“app_id / app_secret are empty: waiting for scripts/setup-
secrets.sh”
```

5. 阶段 D：录入凭证

```
./scripts/setup-secrets.sh
```

脚本依次提示以下各项。密钥类输入不回显，粘贴后直接回车即可；值通过标准输入写入 Secrets Manager，不进入 shell 历史，也不打印到屏幕。

提示	填写	来源
`Webhook URL`	DevOps Agent Webhook URL	2.2
`HMAC signing secret`	Webhook 签名密钥	2.2
`Group bot webhook URL`	**直接回车**（只用应用机器人时不需要）	—
`Signature secret`	**直接回车**	—
`App ID（凭证与基础信息）`	飞书 App ID	3.2
`App Secret（凭证与基础信息）`	飞书 App Secret	3.2
`Verification Token`	**直接回车**（长连接模式不需要）	—

期望输出（只列字段名，不显示值）：

```
aiops-demo/devops-agent-webhook: updated url, secret
aiops-demo/feishu: updated app_id, app_secret; empty: group_webhook_url,
group_webhook_secret, verification_token
```

脚本可以重复运行：回车保留已存的值，输入 - 清空。

检查点：长连接客户端每 30 秒检查一次凭证，录入后 30 秒内连接飞书：

```
aws logs tail /ecs/aiops-demo-feishu-ws --since 5m | grep connected
# 期望：connected to wss://msg-frontier.feishu.cn/ws/v2?<redacted> [conn_id=...]
```

6. 阶段 E：飞书事件订阅与发布

前提：第 5 章的检查点已看到 `connected`。

6.1 选择长连接订阅方式

1. 飞书开发者后台 → 本应用 → **事件与回调** → **事件配置**。
2. 在 **订阅方式** 右侧点编辑，选择 **使用长连接接收事件**，点 **保存**。

飞书要求保存时客户端已在线，否则保存失败（飞书：使用长连接接收事件）。不需要选择“将事件发送至开发者服务器”，也不需要配置请求地址、Encrypt Key、Verification Token。**回调配置**页签不需要配置。

6.2 添加事件

同一页面点 **添加事件**，在 **消息与群组** 分类下勾选 **接收消息 v2.0**（`im.message.receive_v1`），确认添加。如提示缺少权限，按提示开通 3.4 中的权限。

6.3 发布版本

1. **版本管理与发布** → **创建版本**。
2. 填写版本号（如 `1.0.0`）与更新说明；**可用范围** 包含参加演示的成员（飞书：配置应用可用范围）。
3. 保存并 **申请线上发布**。企业开启了应用审核时，等待管理员在管理后台审核通过。

> 权限、事件的任何改动都要发布新版本后才生效。

6.4 把机器人加入演示群

飞书客户端打开演示群 → **群设置** → **群机器人** → **添加机器人**，搜索 3.1 中的应用名并添加。

检查点：在群里 @ 该机器人发送“你好”，约 30–60 秒后机器人在该消息的话题中回复一张标题为“DevOps Agent”的卡片。

```
aws logs tail /ecs/aiops-demo-feishu-ws --since 5m | grep -E 'accepted|ignored'
# 期望：accepted message om_... chat_id=oc_...
```

7. 阶段 F：绑定演示群

1. 从上一步的日志中复制 `chat_id=` 后面以 `oc_` 开头的群 ID。
2. 重新部署，写入群 ID：

```
FEISHU_CHAT_ID=<oc_ 开头的群 ID> AGENT_SPACE_ID="$AGENT_SPACE_ID" ./deploy.sh
# 期望输出：==> chat bot: true, cards via: app bot -> oc_...
```

绑定后：告警、开始调查、根因、恢复卡片都由应用机器人发到这个群；问答只处理这个群的消息，其他群的 @ 会被忽略。之后重跑 `deploy.sh` 不传 `FEISHU_CHAT_ID` 也会沿用这个值。

3. 发送测试卡片：

```
./scripts/test-feishu.sh          # 期望：OK: test card sent via app，群里出现“连通性测试”卡片
```

8. 阶段 G：注册 MCP

8.1 冒烟测试

```
./scripts/test-mcp.sh
```

期望：

- nexus-mock：`initialize`、`tools/list`、`tools/call` 均返回 `HTTP 200`。
- es-mock：先从 Cognito 拿到 OAuth 令牌，再 `initialize`、`tools/list`（`es_search_logs`、`es_error_summary`）、`tools/call` 均返回 HTTP 200。

可选的安全检查（未签名请求应被拒绝）：

```
NEXUS_URL=$(aws cloudformation describe-stacks --stack-name aiops-demo \
  --query "Stacks[0].Outputs[?OutputKey=='NexusMcpUrl'].OutputValue" --output text)
curl -s -o /dev/null -w '%{http_code}\n' -X POST -H 'Content-Type: application/json' \
  -d '{"jsonrpc":"2.0","id":1,"method":"tools/list"}' "$NEXUS_URL"      # 期望 403
```

8.2 注册到 DevOps Agent

```
AGENT_SPACE_ID="$AGENT_SPACE_ID" ./scripts/register-mcp.sh       # 可传 nexus 或 es 只注册一个
```

脚本为 nexus-mock 注册 SigV4 类型、为 es-mock 注册 OAuth 类型的 MCP Server，并把 Agent Space 允许的工具限定为 4 个只读工具。DevOps Agent 注册时会实际连接端点、取令牌并发现工具，注册成功即说明端点与认证可用。es-mock 的 Cognito 客户端密钥经 0600 临时文件传递，不出现在命令行中。

检查点：

```
aws devops-agent list-associations --agent-space-id "$AGENT_SPACE_ID" \
  --query 'associations[].[serviceId, configuration.keys(@)[0]]' --output table
# 期望除 aws、event_channel 外，还有 mcpserversigv4（nexus-mock）与 mcpserver（es-mock）两条
```

> 每次运行都会新建注册。需要重新注册时，先按第 12 章删除旧的注册。

9. 阶段 H：上传 Skill

1. 打包（`SKILL.md` 必须位于 zip 根目录；只能包含文档，不能有 `scripts/` 目录；不超过 6 MB）：

```
mkdir -p dist && (cd skills/order-service-release-rca && zip -q -X -r ../../dist/order-service-release-rca.zip .)
```

2. 打开 Web App（2.1 中 `get-operator-app` 返回的地址，或从控制台进入 Agent Space 的 Web App）→ **Knowledge** 页面 → **Skills** 选项卡 → **Add skill** → **Upload skill**，选择 `dist/order-service-release-rca.zip`。
3. Agent 类型保持默认的 **Generic**（对所有 Agent 生效），确认校验结果后点 **Upload**。

参考：DevOps Agent Skills

修改 Skill 后，可在 Web App 中直接编辑，或删除旧 Skill 后重新上传。

10. 阶段 I：验证与演练

10.1 完整演练

步骤	操作	期望结果	参考耗时
1	`./scripts/inject-fault.sh`	订单服务配置变为 1.4.0、表名 `aiops-demo-orders-v2`	—
2	等待	飞书群收到红色告警卡片	1.5–2 分钟
3	等待	收到蓝色“开始调查”卡片；点按钮可在 Web App 实时查看调查过程	告警后约 15 秒
4	等待	收到绿色根因卡片：1.4.0 把 `ORDERS_TABLE` 指向不存在的表，发布前置迁移未执行，建议回滚	6–8 分钟
5	群里 @机器人 追问，如“回滚到 1.3.2 要改哪些配置？”	话题内回复，可继续追问	30–60 秒
6	`./scripts/rollback.sh`	配置恢复为 1.3.2	—
7	等待	收到绿色告警恢复卡片	1–2 分钟

在 Web App 的调查日志中可以确认：调用了 `es_error_summary` / `es_search_logs` 与 `nexus_list_versions` / `nexus_get_release`，参考了 Skill `order-service-release-rca`，CloudTrail 变更由后台子任务查询。

10.2 演示前检查清单（建议演示前一天）

- [ ] 模拟流量 `"failed": 0`（4.3 检查点）
- [ ] 长连接服务 `runningCount` 为 1，日志有 `connected`
- [ ] `./scripts/test-feishu.sh` 输出 `OK: test card sent via app`
- [ ] `./scripts/test-mcp.sh` 全部 HTTP 200
- [ ] 完整演练一次并记录耗时；保留这次调查记录，现场网络或时间不够时可直接在 Web App 展示
- [ ] 群里 @机器人 能收到回复

DevOps Agent 会参考近期所有相关变更，包括之前演练的注入与回滚。正式演示前几小时内尽量不要再演练，或提前准备说明。执行注入脚本的 IAM 身份会出现在调查结论中，建议用名称清晰的专用角色执行。

10.3 演示当天流程（约 20 分钟）

时间	操作	讲解要点
0:00	展示目标架构图与 Demo 部署架构图	现有拓扑在 AWS 上一一对应，MCP Server 可复用，飞书群只需一个机器人
0:03	运行 `inject-fault.sh`	模拟一次“升级出问题”：配置指向了尚未创建的新表
0:05	群里出现告警卡片、开始调查卡片	告警自动转交 DevOps Agent，无需人工介入
0:06	打开 Web App 查看调查过程	每一步的工具调用与依据：ES 日志、CloudTrail 变更、Nexus 发布说明
0:12	群里出现根因卡片	根因、证据链、处置建议；Agent 只给建议，不执行变更
0:14	@机器人 追问	话题内多轮对话
0:16	运行 `rollback.sh`，等待恢复卡片	处置由人执行，闭环完成
0:19	收尾	安全设计：无未认证的公网入口、只读调查、全程可审计

11. 日常运维

操作	命令
暂停长连接（保留资源）	`FEISHU_WS_DESIRED_COUNT=0 AGENT_SPACE_ID="$AGENT_SPACE_ID" ./deploy.sh`
恢复长连接	`FEISHU_WS_DESIRED_COUNT=1 AGENT_SPACE_ID="$AGENT_SPACE_ID" ./deploy.sh`
停用 / 恢复模拟流量	`TRAFFIC_ENABLED=false ./deploy.sh` / `TRAFFIC_ENABLED=true ./deploy.sh`（均需带 `AGENT_SPACE_ID`）
查看长连接日志	`aws logs tail /ecs/aiops-demo-feishu-ws --follow`
查看问答日志	`aws logs tail /aws/lambda/aiops-demo-chat-worker --follow`
查看告警与卡片日志	`aws logs tail /aws/lambda/aiops-demo-alert-forwarder --follow`
更换飞书或 Webhook 凭证	`./scripts/setup-secrets.sh`（回车保留其他值）；长连接客户端需重启任务后使用新凭证：`aws ecs update-service --cluster aiops-demo --service aiops-demo-feishu-ws --force-new-deployment`
修改代码后更新	重新运行 `deploy.sh`；长连接客户端代码变化会自动滚动更新

费用参考：长连接任务约 7.2 美元/月，其公网 IPv4 约 3.65 美元/月；每次调查约 3–4 美元；其余资源按量计费，演示规模下很小。

12. 清理

1. 删除栈、制品桶与 AgentCore 自建的日志组（需输入 yes 确认，不可恢复）：

```
./scripts/destroy.sh
```

2. 删除 DevOps Agent 中的 MCP 注册：

```
aws devops-agent list-associations --agent-space-id "$AGENT_SPACE_ID" \
  --query 'associations[].[associationId,serviceId]' --output table
```

```
aws devops-agent disassociate-service --agent-space-id "$AGENT_SPACE_ID" --association-id
<MCP 关联 ID>
aws devops-agent deregister-service --service-id <MCP serviceId>
```

3. 控制台中删除 HMAC Webhook；Agent Space 为本次演示专用时一并删除。

4. 飞书：在演示群中移除机器人；在开发者后台停用或删除应用。

5. 本机：删除项目根目录下的 `.local/`、`dist/`、`build/` 与本机创建的虚拟环境（`.venv*`）。

13. 故障排查

现象	可能原因	处理
飞书保存“使用长连接接收事件”失败	客户端不在线	确认第 5 章检查点出现 `connected`；ECS 服务 `runningCount` 为 1
长连接日志一直显示 waiting	飞书凭证未录入或录错 Secret	重新运行 `setup-secrets.sh`，确认 `AWS_PROFILE` 指向部署栈的账号
@机器人 后日志里没有任何记录	应用版本未发布；未添加“接收消息 v2.0”；@ 的不是该应用机器人；可用范围不含发送者	按 6.2–6.4 检查，改动后重新发布版本
日志显示 `ignored: chat not allowed`	消息来自未绑定的群	在绑定的演示群里提问，或按第 7 章重新绑定
日志显示 `accepted` 但没有回复	问答 Worker 出错	查看 `/aws/lambda/aiops-demo-chat-worker`；常见为飞书返回 230002（机器人不在群）或缺少“以应用的身份发消息”权限
`test-feishu.sh` 失败	应用凭证错误；机器人不在群（230002）	核对 App ID / Secret；把机器人加入群
使用群自定义机器人时卡片发送失败	19021 签名错误；19024 开启了自定义关键词；19007 机器人不可用	推荐改用应用机器人发送（第 7 章）
告警后没有调查	Webhook URL 或签名密钥错误（告警转换日志中出现 4xx）	重新创建 HMAC Webhook 并重新录入
同一告警再次触发没有新调查	15 分钟收敛窗口内，或上一次事件尚未恢复	等告警恢复后再注入
`register-mcp.sh` 被拒绝	调用者缺少 `aidevops` 注册权限，或账号 / 组织策略限制注册自定义 MCP	检查 IAM 权限与组织策略
`register-mcp.sh` 连接或发现工具失败	MCP 端点或认证不可用	先运行 `test-mcp.sh` 定位
调查结论中出现“操作被取消”的证据缺口	DevOps Agent 自主调查只执行只读操作，部分调用被判定为需审批	在 Skill 中写明只读替代操作（参考本 Demo 的 Skill 第 2、4 步）
ECS 部署失败并自动回滚	子网没有互联网出站路由；无法拉取镜像或访问 S3	使用默认 VPC，或通过 `FEISHU_WS_VPC` / `FEISHU_WS_SUBNETS` 指定有出站路由的子网
模拟流量没有请求	已停用流量	`TRAFFIC_ENABLED=true` 重新部署

附录：项目文件清单（`aiops-aws-demo/`）

路径	说明
`README.md`	项目说明与快速开始
`template.yaml`	CloudFormation 模板
`deploy.sh`	打包与部署
`scripts/`	凭证录入、MCP 注册与测试、飞书测试、故障注入与回滚、清理
`skills/order-service-release-rca/SKILL.md`	排障 Skill
`src/`	Lambda、长连接客户端与 MCP Server 代码
`tests/test_units.py`	单元测试：`python3.12 -m venv .venv && .venv/bin/pip install boto3==1.43.113 && .venv/bin/python -m unittest discover -s tests`
`docs/`	本文档、方案说明、设计文档、架构图源文件（`diagrams/*.svg`）与文档生成脚本